

Inertial Filter

Component Design Document

1 Description

Inertial attitude filter. Estimates the body attitude and body rate from the star tracker attitude and rate with a square root unscented Kalman filter, and publishes the estimate for the guidance and control algorithms. A diagnostics packet with the full filter state, covariance, and residuals is produced at a commanded period. The filter keeps its own time base, which restarts at the tick after it is built or reset, so the star tracker time tag and the tick time must be on the same clock. Wraps the InertialFilterAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 11
- **Number of Invokee Connectors** - 5
- **Number of Invoker Connectors** - 6
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - 1
- **Number of Parameters** - 7
- **Number of Events** - 2
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 1
- **Number of Packets** - 1

3.2 Diagram

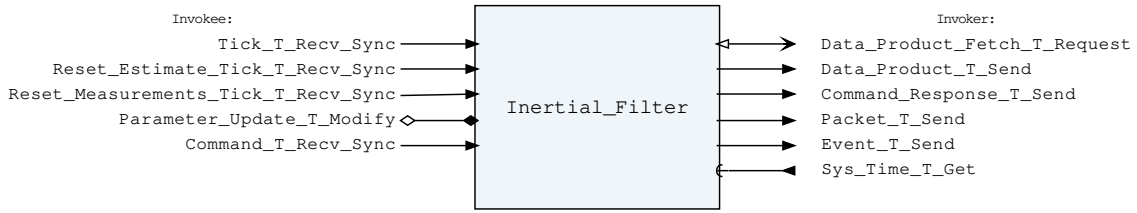


Figure 1: Inertial Filter component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Inertial Filter Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Reset_Estimate_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Reset_Measurements_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_Update_T_Modify	modify	Parameter_Update.T	-	1
Command_T_Recv_Sync	recv_sync	Command.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - Run the filter up to the current time, folding in a fresh star tracker reading when there is one.
- **Reset_Estimate_Tick_T_Recv_Sync** - Re-seed the filter state and covariance from the configured initial values and clear the pending measurements and residuals. The filter's time base restarts at the next tick. TODO verify with the GNC team whether this reset is needed at all.
- **Reset_Measurements_Tick_T_Recv_Sync** - Clear the pending measurements and residuals, keeping the filter state and covariance. The filter's time base restarts at the next tick.
- **Parameter_Update_T_Modify** - The parameter update connector.
- **Command_T_Recv_Sync** - This is the command receive connector.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Inertial Filter Invoker Connectors

Name	Kind	Type	Return_Type	Count
------	------	------	-------------	-------

Data_Product_ Fetch_T_Request	request	Data_Product_ Fetch.T	Data_Product_ Return.T	1
Data_Product_T_ Send	send	Data_Product.T	-	1
Command_ Response_T_Send	send	Command_ Response.T	-	1
Packet_T_Send	send	Packet.T	-	1
Event_T_Send	send	Event.T	-	1
Sys_Time_T_Get	get	-	Sys_Time.T	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector
- **Command_Response_T_Send** - This connector is used to register and respond to the component's commands.
- **Packet_T_Send** - The diagnostics packet is sent out of this connector.
- **Event_T_Send** - Events are sent out of this connector.
- **Sys_Time_T_Get** - The system time is retrieved via this connector.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Inertial Filter Set Id Bases Parameters

Name	Type
Command_Id_Base	Command_Types.Command_Id_Base
Data_Product_Id_Base	Data_Product_Types.Data_Product_Id_Base
Event_Id_Base	Event_Types.Event_Id_Base
Packet_Id_Base	Packet_Types.Packet_Id_Base
Parameter_Id_Base	Parameter_Types.Parameter_Id_Base

Parameter Descriptions:

- **Command_Id_Base** - The value at which the component's command identifiers begin.
- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Event_Id_Base** - The value at which the component's event identifiers begin.
- **Packet_Id_Base** - The value at which the component's unresolved packet identifiers begin.
- **Parameter_Id_Base** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Inertial Filter Map Data Dependencies Parameters

Name	Type
Star_Tracker_Attitude_Id	Data_Product_Types.Data_Product_Id
Star_Tracker_Attitude_Stale_Limit	Ada.Real_Time.Time_Span

Parameter Descriptions:

- **Star_Tracker_Attitude_Id** - The external data product ID used to fetch the value for the 'Star_Tracker_Attitude' data dependency.
- **Star_Tracker_Attitude_Stale_Limit** - The stale limit used for the 'Star_Tracker_Attitude' data dependency. If the fetched data product is older than the given `Time_Span` then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the inertial filter with the default parameter values, which seed the filter state and covariance. The diagnostics packet is off until its period is commanded. The `init` subprogram takes no parameters.

3.6 Commands

Commands for the Inertial Filter component.

Table 5: Inertial Filter Commands

Local ID	Command Name	Argument Type
0	Set_Diagnostics_Packet_Period	Packed_U16.T

Command Descriptions:

- **Set_Diagnostics_Packet_Period** - Set the period of the diagnostics packet in ticks. Zero turns the packet off.

3.7 Parameters

Parameters for the Inertial Filter component. The process noise and the initial covariance are given by their diagonals, since the full matrices do not fit in a parameter. TODO verify with the GNC

team that diagonal matrices are enough.

Table 6: Inertial Filter Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Alpha	Packed_F64.T	(Value=>0.02)
0x0001 (1)	Beta	Packed_F64.T	(Value=>2.0)
0x0002 (2)	Process_Noise_Diagonal	Packed_F64x6.T	[1.0E-14, 1.0E-14, 1.0E-14, 1.0E-16, 1.0E-16, 1.0E-16]
0x0003 (3)	Initial_State	Packed_F64x6.T	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
0x0004 (4)	Initial_Covariance_Diagonal	Packed_F64x6.T	[1.0E-4, 1.0E-4, 1.0E-4, 1.0E-4, 1.0E-4, 1.0E-4]
0x0005 (5)	St_Measurement_Noise_Std	Packed_F64.T	(Value=>1.0E-4)
0x0006 (6)	Rate_Measurement_Noise_Std	Packed_F64.T	(Value=>0.001)

Parameter Descriptions:

- **Alpha** - [-] Sigma point spread. Must be in (0, 1].
- **Beta** - [-] Prior knowledge tunable. Must be in [0, 2].
- **Process_Noise_Diagonal** - [-] Diagonal of the N x N process noise covariance, one entry per state: the attitude MRP followed by the body rate. Every entry must be finite and not negative.
- **Initial_State** - [-] State the filter is seeded with, the attitude MRP followed by the body rate in rad/s. Must be finite.
- **Initial_Covariance_Diagonal** - [-] Diagonal of the N x N covariance the filter is seeded with, one entry per state. Every entry must be finite and not negative.
- **St_Measurement_Noise_Std** - [-] Standard deviation of the star tracker attitude measurement noise. Must not be negative.
- **Rate_Measurement_Noise_Std** - [rad/s] Standard deviation of the rate measurement noise. Must not be negative.

3.8 Events

Events for the Inertial Filter component.

Table 7: Inertial Filter Events

Local ID	Event Name	Parameter Type
0	Diagnostics_Packet_Period_Set	Packed_U16.T
1	Invalid_Command_Received	Invalid_Command_Info.T

Event Descriptions:

- **Diagnostics_Packet_Period_Set** - The diagnostics packet period was set via command, in ticks. Zero turns the packet off.
- **Invalid_Command_Received** - A command was received with invalid arguments.

3.9 Data Products

Data products for the Inertial Filter component.

Table 8: Inertial Filter Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Attitude_Estimate	Nav_Att_Output.T

Data Product Descriptions:

- **Attitude_Estimate** - Estimated body attitude and body rate, stamped with the time the filter advanced to. The sun direction is not estimated by this filter and is zero.

3.10 Data Dependencies

Data dependencies for the Inertial Filter component.

Table 9: Inertial Filter Data Dependencies

Data Dependency Name	Type
Star_Tracker_Attitude	St_Att.T

Data Dependency Descriptions:

- **Star_Tracker_Attitude** - Star tracker attitude and body rate in the body frame, with the time tag of the measurement on the system clock. Both are fed to the filter as measurements when the time tag has advanced since the last reading consumed.

3.11 Packets

Packets for the Inertial Filter component.

Table 10: Inertial Filter Packets

Local ID	Packet Name	Type
0x0000 (0)	Filter_Diagnostics	Inertial_Filter_Output.T

Packet Descriptions:

- **Filter_Diagnostics** - The full filter snapshot after the update, the state, the covariance, and the residuals of each measurement kind. Sent every diagnostics packet period, in ticks, as set by command.

3.12 Faults

The Inertial Filter component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Inertial_Filter_Tests* Test Suite

This is a unit test suite for the Inertial Filter component

Test Descriptions:

- **Test** - Feed a constant star tracker attitude and check the estimate converges to it while the covariance shrinks, as in the algorithm's Python reference test, to ensure the Ada to C to C++ integration is sound.
- **Test_Rate_Update** - Feed a constant body rate about one axis with the attitude that rotation gives, and check the rate estimate converges to it while the attitude tracks the truth.
- **Test_Propagation** - Ensure a star tracker product whose time tag has not advanced is not fed to the filter, so the estimate holds and the covariance grows under the process noise.
- **Test_Reset** - Ensure the estimate reset re-seeds the state and covariance while the measurements reset keeps them.
- **Test_Reading_Before_Time_Base** - Ensure a star tracker reading stamped before the time base restarted by a reset is dropped, and a later one is applied.
- **Test_Time_Anomalies** - Ensure a reading stamped after the tick waits until the tick reaches it, and a tick before the time base restarts it, without either shutting out the readings that follow.
- **Test_Diagnostics_Packet** - Ensure the diagnostics packet is sent every commanded number of ticks, that a period of zero turns it off, and that the command is reported.
- **Test_Parameter_Update** - Ensure a parameter update rebuilds the filter from the new configuration, restarting the estimate from the configured seed.
- **Test_Invalid_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **Test_Invalid_Command** - Ensure a malformed command is reported rather than acted on.
- **Test_Invalid_Data_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Command.T:

Generic command packet for holding arbitrary commands

Table 11: Command Packed Record : 2080 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Command_Header.T	-	40	0	39	-
Arg_Buffer	Command_Types. Command_Arg_Buffer_Type	-	2040	40	2079	Header.Arg_Buffer_Length

Field Descriptions:

- **Header** - The command header
- **Arg_Buffer** - A buffer that contains the command arguments

Command_Header.T:

Generic command header for holding arbitrary commands

Table 12: Command_Header Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types. Command_Source_Id	0 to 65535	16	0	15
Id	Command_Types. Command_Id	0 to 65535	16	16	31
Arg_Buffer_Length	Command_Types. Command_Arg_Buffer_Length_Type	0 to 255	8	32	39

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Id** - The command identifier
- **Arg_Buffer_Length** - The number of bytes used in the command argument buffer

Command_Response.T:

Record for holding command response data.

Table 13: Command_Response Packed Record : 56 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types.Command_Source_Id	0 to 65535	16	0	15
Registration_Id	Command_Types.Command_Registration_Id	0 to 65535	16	16	31
Command_Id	Command_Types.Command_Id	0 to 65535	16	32	47

Status	Command_Enums. Command_ Response_ Status.E	0 => Success 1 => Failure 2 => Id_Error 3 => Validation_Error 4 => Length_Error 5 => Dropped 6 => Register 7 => Register_Source	8	48	55
--------	---	--	---	----	----

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Registration_Id** - The registration ID. An ID assigned to each registered component at initialization.
- **Command_Id** - The command ID for the command response.
- **Status** - The command execution status.

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 14: Data_Product Packed Record : 1096 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_Header.T	-	72	0	71	-
Buffer	Data_Product_Types.Data_Product_Buffer_Type	-	1024	72	1095	Header. Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 15: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 16: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 17: Data_Product_Return Packed Record : 1104 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Event.T:

Generic event packet for holding arbitrary events

Table 18: Event Packed Record : 328 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Event_Header.T	-	72	0	71	-
Param_Buffer	Event_Types. Parameter_ Buffer_Type	-	256	72	327	Header.Param_ Buffer_Length

Field Descriptions:

- **Header** - The event header

- **Param_Buffer** - A buffer that contains the event parameters

Event_Header.T:

Generic event packet for holding arbitrary events

Table 19: Event_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Event_Types.Event_Id	0 to 65535	16	48	63
Param_Buffer_Length	Event_Types.Parameter_Buffer_Length_Type	0 to 32	8	64	71

Field Descriptions:

- **Time** - The timestamp for the event.
- **Id** - The event identifier
- **Param_Buffer_Length** - The number of bytes used in the param buffer

Inertial_Filter_Output.T:

Snapshot of the inertial filter after an update. Maps to InertialFilterOutput_c in the inertialFilter C shim, which returns it by value.

Table 20: Inertial_Filter_Output Packed Record : 3856 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
State	Packed_F64x6.T	-	384	0	383
Covariance	Packed_F64x36.T	-	2304	384	2687
St_Att_Residuals	Inertial_Filter_Residuals.T	-	584	2688	3271
Rate_Residuals	Inertial_Filter_Residuals.T	-	584	3272	3855

Field Descriptions:

- **State** - [-] Estimated state, the body attitude MRP followed by the body rate in rad/s.
- **Covariance** - [-] N x N state covariance in row major order.
- **St_Att_Residuals** - Star tracker attitude residuals of this update.
- **Rate_Residuals** - Rate residuals of this update.

Inertial_Filter_Residuals.T:

Residuals of one measurement kind from one inertial filter update. Maps to InertialResiduals_c in the inertialFilter C shim.

Table 21: Inertial_Filter_Residuals Packed Record : 584 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
------	------	-------	-------------	-----------	---------

Valid	Boolean	0 => False 1 => True	8	0	7
Observation	Packed_F64x3.T	-	192	8	199
Pre_Fit	Packed_F64x3.T	-	192	200	391
Post_Fit	Packed_F64x3.T	-	192	392	583

Field Descriptions:

- **Valid** - Whether this measurement fired and was applied this update. The other fields are meaningful only when it did. Mirrors the C99 bool in the shim's struct.
- **Observation** - [-] Measured value.
- **Pre_Fit** - [-] Residual before the update.
- **Post_Fit** - [-] Residual after the update.

Invalid_Command_Info.T:

Record for holding information about an invalid command

Table 22: Invalid_Command_Info Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Command_Types. Command_Id	0 to 65535	16	0	15
Errant_Field_Number	Interfaces. Unsigned_32	0 to 4294967295	32	16	47
Errant_Field	Basic_Types.Poly_ Type	-	64	48	111

Field Descriptions:

- **Id** - The command Id received.
- **Errant_Field_Number** - The field that was invalid. 1 is the first field, 0 means unknown field, 2**32 means that the length field of the command was invalid.
- **Errant_Field** - A polymorphic type containing the bad field data, or length when Errant_Field_Number is 2**32.

Nav_Att_Output.T:

Table 23: Nav_Att_Output Packed Record : 352 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time_Tag	Long_Float	-inf to inf	64	0	63
Sigma_Bn	Packed_F32x3.T	-	96	64	159
Omega_Bn_B	Packed_F32x3.T	-	96	160	255
Veh_Sun_Pnt_Bdy	Packed_F32x3.T	-	96	256	351

Field Descriptions:

- **Time_Tag** - [s] Time tag of navigation solution
- **Sigma_Bn** - MRP attitude of body w.r.t. inertial frame
- **Omega_Bn_B** - [rad/s] Angular velocity of body w.r.t. inertial, body frame components

- **Veh_Sun_Pnt_Bdy** - Sun direction unit vector in body frame

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 24: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Packed_F64.T:

Single component record for holding packed 64-bit floating point number.

Table 25: Packed_F64 Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Long_Float	-inf to inf	64	0	63

Field Descriptions:

- **Value** - The 64-bit floating point number.

Packed_F64x3.T:

Packed 3 element vector of 64-bit floats.

Table 26: Packed_F64x3 Packed Array : 192 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	3	192

Packed_F64x36.T:

Packed 36 element vector of 64-bit floats.

Table 27: Packed_F64x36 Packed Array : 2304 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	36	2304

Packed_F64x6.T:

Packed 6 element vector of 64-bit floats.

Table 28: Packed_F64x6 Packed Array : 384 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	6	384

Packed_U16.T:

Single component record for holding packed unsigned 16-bit value.

Table 29: Packed_U16 Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces. Unsigned_16	0 to 65535	16	0	15

Field Descriptions:

- **Value** - The 16-bit unsigned integer.

Packet.T:

Generic packet for holding arbitrary data

Table 30: Packet Packed Record : 8128 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Packet_ Header.T	-	96	0	95	-
Buffer	Packet_ Types.Packet_ Buffer_Type	-	8032	96	8127	Header. Buffer_Length

Field Descriptions:

- **Header** - The packet header
- **Buffer** - A buffer that contains the packet data

Packet_Header.T:

Generic packet header for holding arbitrary data

Table 31: Packet_Header Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Packet_Types. Packet_Id	0 to 65535	16	48	63
Sequence_Count	Packet_Types. Sequence_Count_Mod_ Type	0 to 16383	16	64	79
Buffer_Length	Packet_Types. Packet_Buffer_ Length_Type	0 to 1004	16	80	95

Field Descriptions:

- **Time** - The timestamp for the packet item.

- **Id** - The packet identifier
- **Sequence_Count** - Packet Sequence Count
- **Buffer_Length** - The number of bytes used in the packet buffer

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 32: Parameter Packed Record : 792 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_Header.T	-	24	0	23	-
Buffer	Parameter_Types. Parameter_Buffer_Type	-	768	24	791	Header.Buffer_Length

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 33: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 34: Parameter_Update Packed Record : 824 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_Types. Parameter_Table_Id	0 to 65535	16	0	15	-

Operation	Parameter_Enums. Parameter_Operation_Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate	8	16	23	-
Status	Parameter_Enums. Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter.T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

St_Att.T:

Table 35: St_Att Packed Record : 256 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time_Tag	Interfaces. Unsigned_64	0 to 18446744073709551615	64	0	63
Sigma_Bn	Packed_ F32x3.T	-	96	64	159
Omega_Bn_B	Packed_ F32x3.T	-	96	160	255

Field Descriptions:

- **Time_Tag** - [ns] Time tag associated with measurement
- **Sigma_Bn** - [-] MRP from inertial to body frame
- **Omega_Bn_B** - [rad/s] Body-frame angular velocity w.r.t. inertial

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 36: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31

Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47
------------	----------------------------	------------	----	----	----

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of $1/(2^{16})$ sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 37: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Command_Enums.Command_Response_Status.E:

This status enumeration provides information on the success/failure of a command through the command response connector.

Table 38: Command_Response_Status Literals:

Name	Value	Description
Success	0	Command was passed to the handler and successfully executed.
Failure	1	Command was passed to the handler not successfully executed.
Id_Error	2	Command id was not valid.
Validation_Error	3	Command parameters were not successfully validated.
Length_Error	4	Command length was not correct.
Dropped	5	Command overflowed a component queue and was dropped.
Register	6	This status is used to register a command with the command routing system.
Register_Source	7	This status is used to register command sender's source id with the command router for command response forwarding.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 39: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 40: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 41: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.